

Practical Insights into Data Analysis and Machine Learning

PROGRAM - 7

Develop a program to load the Titanic dataset. Split the data into training and test sets. Train a decision tree classifier. Visualize the tree structure. Evaluate accuracy, precision, recall, and F1-score.

Objective

To implement classifier using a Decision Tree model and visualize the tree.

7. Introduction

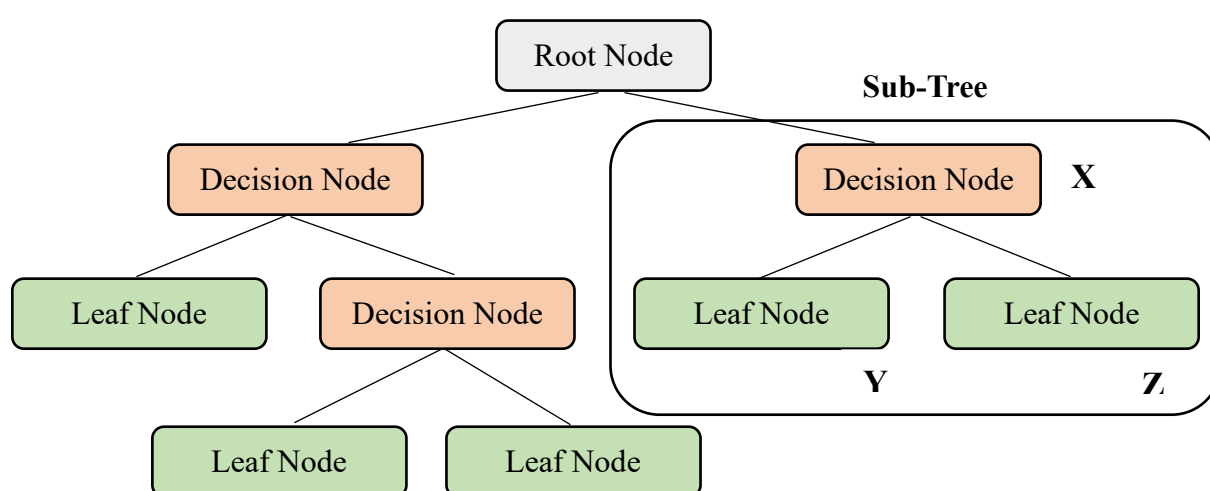
The Titanic dataset is a classic benchmark in machine learning, used to predict the survival of passengers based on features such as age, sex, class, and more. In this program, we aim to build a Decision Tree classifier to model survival outcomes. By splitting the dataset into training and testing sets, we train the model to learn patterns from the data and then evaluate its predictive performance using key classification metrics. Additionally, the tree structure is visualized to interpret how decisions are made based on passenger.

7.1 Decision Tree

Decision Trees (DTs) are a non-parametric supervised learning method used for classification and regression. The goal is to create a model that predicts the value of a target variable by learning simple decision rules inferred from the data features.

Key Terms in Decision Trees:

- **Root Node:** The starting point of the tree.
- **Splitting:** Dividing a node into multiple sub-nodes.
- **Decision Node:** A node that splits into more sub-nodes.
- **Leaf Node:** A node that does not split further and represents an outcome.
- **Pruning:** Removing sub-nodes to simplify the tree.
- **Branch:** A part of the tree with multiple nodes



How Decision Trees Operate:

- A decision tree looks like an upside-down tree, starting with a root node.
- From the root node, the tree splits into decision nodes, and these further split until they reach leaf nodes, which show the outcomes.
- Each decision node represents a condition, and the branches represent the possible answers

Types of decision trees in machine learning

Decision trees in machine learning can either be classification trees or regression trees.

1. **Classification trees** determine whether an event happened or not. This involves a “yes” or “no” outcome. It deals with categories.
Example: Is an animal a reptile or mammal?
2. **Regression trees** predict continuous values based on previous data or information sources. It deals with numerical outcomes.
Example: Predicting house prices based on features like size and location.

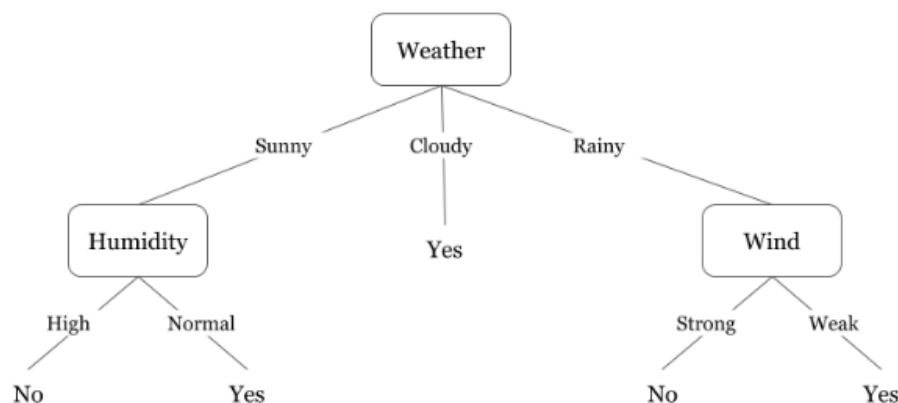


Figure: A decision tree for the concept *PlayTennis*

Variants of Decision Trees

1. **ID3:** Ross Quinlan is credited within the development of ID3, which is shorthand for “*Iterative Dichotomiser 3*.” This algorithm uses *entropy and information gain* as metrics to evaluate candidate splits.
2. **C4.5:** This algorithm is considered a later iteration of ID3, which was also developed by Quinlan. It can use *gain ratios* to evaluate split points within the decision trees.
3. **CART:** The term, CART, is an abbreviation for “*Classification and Regression Trees*” and was introduced by Leo Breiman. This algorithm typically utilizes *Gini impurity* to identify the ideal attribute to split on.

Entropy measures the impurity or uncertainty in a dataset.

- Higher entropy = more disorder (mixed classes),
- Lower entropy = more pure (mostly one class).

Gini Index also measures impurity, but it's faster to compute and often used in CART. Gini Index measures how often a randomly chosen element would be incorrectly classified.

Information Gain tells us how much entropy is reduced by a split. It's the difference between the parent entropy and the weighted average of child entropies.

ID3 uses **Information Gain (IG)** based on **Entropy**.

◆ Entropy:

Entropy measures the impurity (or uncertainty) in the dataset.

$$H(S) = - \sum_{i=1}^c p_i \log_2(p_i)$$

- S : dataset
- c : number of classes
- p_i : proportion of samples in class i

◆ Information Gain:

$$IG(S, A) = H(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} H(S_v)$$

- A : feature
- S_v : subset of S where feature A has value v
- The split with the **highest information gain** is chosen.

◆ Gain Ratio:

$$\text{GainRatio}(S, A) = \frac{IG(S, A)}{\text{SplitInformation}(S, A)}$$

◆ Split Information:

$$\text{SplitInformation}(S, A) = - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} \log_2 \left(\frac{|S_v|}{|S|} \right)$$

- C4.5 chooses the feature with the **highest gain ratio**.

◆ Gini Impurity:

$$\text{Gini}(S) = 1 - \sum_{i=1}^c (p_i)^2$$

- The lower the Gini, the purer the node.

◆ Gini Gain (Decrease in Impurity):

$$\Delta \text{Gini}(S, A) = \text{Gini}(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} \text{Gini}(S_v)$$

CART chooses the split that **minimizes the weighted Gini impurity**.

🔗 Note: CART always produces binary trees, unlike ID3 and C4.5 which can create multiway splits.

7.2 Program

```
import pandas as pd
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
import matplotlib.pyplot as plt

# Load the Titanic Dataset
data = sns.load_dataset('titanic')

# 2. Preprocess the dataset
# Select relevant features and drop rows with missing values
features = ['pclass', 'sex', 'age', 'sibsp', 'parch', 'fare', 'embarked']
data = data[features + ['survived']].dropna()

# Convert categorical columns to numerical
data['sex'] = data['sex'].map({'male': 0, 'female': 1})
data['embarked'] = data['embarked'].map({'C': 0, 'Q': 1, 'S': 2})

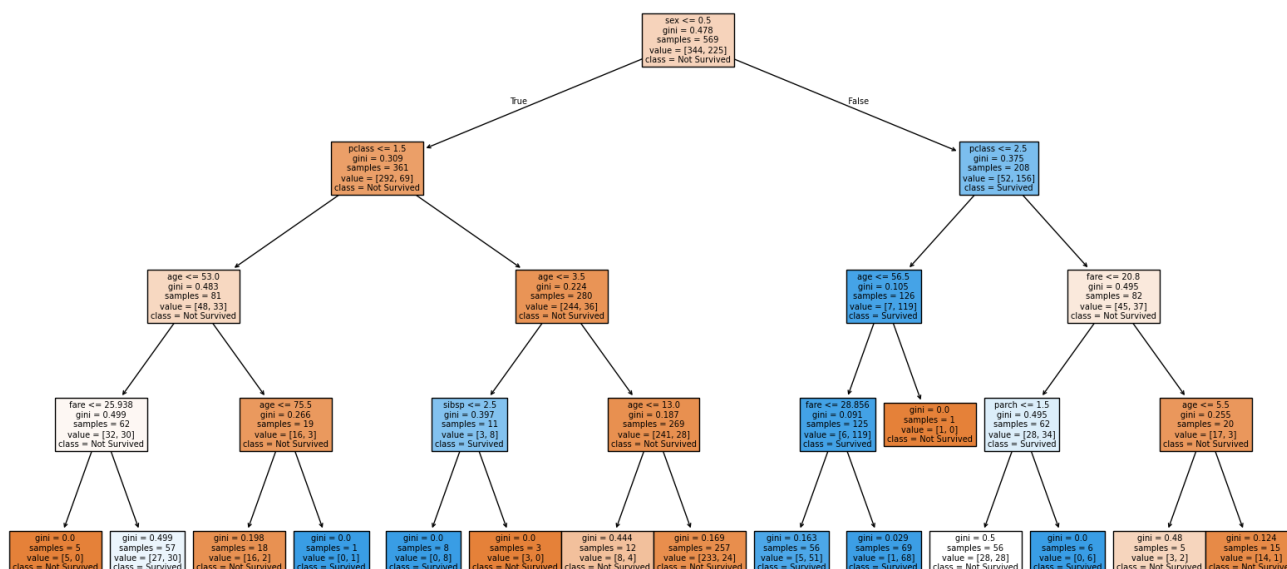
# Define features and target
X = data[features]
y = data['survived']

# 3. Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                    test_size=0.2, random_state=42)

# 4. Train the Decision Tree Classifier
clf = DecisionTreeClassifier(criterion='gini', max_depth=4,
                             min_samples_split=2, random_state=42)
clf.fit(X_train, y_train)

# 5. Visualize the Decision Tree
plt.figure(figsize=(20,10))
plot_tree(clf, feature_names=features,
          class_names=['Not Survived', 'Survived'], filled=True)
plt.title("Decision Tree for Titanic Dataset")
plt.show()
```

Decision Tree for Titanic Dataset



6. Evaluate the model

```
y_pred = clf.predict(X_test)
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
precision = precision_score(y_test, y_pred)
```

```
recall = recall_score(y_test, y_pred)
```

```
f1 = f1_score(y_test, y_pred)
```

```
print(f"Accuracy: {accuracy:.2f}")
```

```
print(f"Precision: {precision:.2f}")
```

```
print(f"Recall: {recall:.2f}")
```

```
print(f"F1 Score: {f1:.2f}")
```

Accuracy: 0.71

Precision: 0.72

Recall: 0.54

F1 Score: 0.62

Viva Questions

- What is a Decision Tree?
- Explain the difference between a root node, decision node, and leaf node.
- What are classification trees and regression trees?
- What is the purpose of splitting a dataset into training and test sets?
- Why do we use the Titanic dataset in machine learning?
- What is overfitting in the context of decision trees?
- How can we control overfitting in a decision tree?
- What is the role of max_depth in DecisionTreeClassifier?
- What is the Gini Index? How is it different from entropy?
- What does a Gini value of 0 mean in a node?
- What is Information Gain?
- Why does CART use Gini Index while ID3 uses entropy?
- What is the difference between ID3, C4.5, and CART?
- What are the advantages and limitations of Decision Trees?
- Explain what pruning is and why it is important.